PHỤC DỰNG MÔ HÌNH KHUẾCH TÁN KẾT HỢP MẠNG NƠ-RON ĐỒ THỊ CHO BÀI TOÁN ĐỊNH TUYẾN TRONG MẠNG SAGSIN

Báo cáo đồ án PBL4 — phần mô hình sinh và mô hình đối chứng
Nhóm 3 người: TV1 (mạng, dữ liệu) · TV2 (mô hình khuếch tán)
TV3 (trực quan hóa, GNN, báo cáo)
Tháng 9/2026

Tóm tắt

Báo cáo trình bày việc phục dựng một mô hình khuếch tán (diffusion model, DM) kết hợp mạng nơ-ron đồ thị (GNN) để chọn đường đi trong mạng tích hợp không gian – trên không – mặt đất (SAGSIN) gồm 73 nút: 50 vệ tinh quỹ đạo thấp, 5 HAPS, 5 UAV, 5 trạm cổng và 8 người dùng. Mô hình học bắt chước thuật toán Dijkstra có quy tắc chuyển tiếp, quyết định từng bước nút kế tiếp (next-hop) cho tới khi tới đích. Để biết từng thành phần đóng góp gì, nhóm xây thêm mô hình đối chứng dùng cùng GNN nhưng thay diffusion bằng bộ phân loại softmax, và đo theo lưới 2 × 2 (có hoặc không có GNN × diffusion hoặc softmax).

Trên tập test gồm 2.700 đường ở các snapshot chưa thấy khi huấn luyện, GNN 4 lớp kết hợp diffusion chọn đúng next-hop 99,3%, tới đích 100% và có độ trễ chỉ cao hơn Dijkstra 0,2%. Bộ phân loại softmax với cùng GNN đạt cùng 99,3%. GNN đóng góp khoảng 16–18 điểm, còn diffusion không làm tăng độ chính xác. Việc đổi cách huấn luyện diffusion từ dự đoán nhiễu (MSE) sang dự đoán thẳng nghiệm y0 (cross-entropy) nâng bản không GNN từ 72,6% lên 81,5% và cho phép chỉ cần 1–2 bước khử nhiễu. Vai trò thật của diffusion là một bộ sinh quyết định có điều kiện, có núm điều khiển mức bám điều kiện (CFG) và độ ngẫu nhiên (η). Khi được huấn luyện bằng nhãn nhiều tuyến gần tối ưu, mô hình sinh sẵn đường dự phòng cho 49,5–62% tình huống hỏng một liên kết, nhưng softmax lấy mẫu có nhiệt độ đạt tương đương hoặc tốt hơn với chi phí thấp hơn. Ở quy mô 73 nút, mọi mô hình học đều chậm hơn Dijkstra khi chạy trên CPU.

1. Giới thiệu

Mạng SAGSIN kết nối ba tầng: vệ tinh quỹ đạo thấp (LEO) ở tầng không gian, các nền tảng bay tầm cao (HAPS) và máy bay không người lái (UAV) ở tầng không trung, cùng trạm cổng và người dùng ở mặt đất [11]. Vì vệ tinh chuyển động liên tục, cấu hình liên kết thay đổi theo từng thời điểm, nên bảng định tuyến phải được tính lại thường xuyên. Các nghiên cứu gần đây dùng mô hình học sâu, trong đó có mô hình khuếch tán, để sinh lời giải cho bài toán tổ hợp trên đồ thị [7]. Đề tài của nhóm là phục dựng một mô hình khuếch tán cho bài toán định tuyến trong SAGSIN, đánh giá nó một cách trung thực và làm rõ vai trò của từng thành phần.

Đóng góp của báo cáo gồm: (i) một quy trình dữ liệu và mô hình chạy được với số nút bất kỳ, gồm GNN mã hóa đồ thị và bộ khử nhiễu chấm điểm từng nút ứng viên; (ii) một cải tiến cách tham số hóa diffusion cho lựa chọn rời rạc, dự đoán thẳng y0 và học bằng cross-entropy; (iii) thí nghiệm đối chứng 2 × 2 tách đóng góp của GNN và của

diffusion; (iv) đánh giá khả năng sinh nhiều tuyến và đường dự phòng; (v) so sánh với DIFUSCO [7] để chỉ ra khi nào diffusion thật sự có lợi thế.

2. Bài toán và mô hình mạng

2.1. Mạng SAGSIN mô phỏng

Dữ liệu do TV1 sinh bằng `oracle_labeler_sagsin.py`. Quỹ đạo 50 vệ tinh Starlink được tính từ bộ phần tử quỹ đạo TLE. 5 HAPS đặt ở độ cao 20 km trên các vùng Bắc, Trung, Nam, Tây Nguyên và Đồng bằng sông Cửu Long. 5 UAV bay tuần tra quanh Vinh, Nha Trang, Cần Thơ, Lào Cai và Quy Nhơn. 5 trạm cổng và 8 người dùng ở mặt đất. Tổng cộng N = 73 nút. Mạng được chụp 600 lần, mỗi lần cách nhau 20 giây (200 phút). Mỗi snapshot t cho một ma trận trễ W_t có kích thước 73 × 73, trong đó $W_t(u,v)$ là độ trễ truyền (ms) nếu có liên kết và bằng −1 nếu không. Trung bình mỗi nút có 5,2 liên kết, mỗi snapshot có khoảng 380 cạnh có hướng, và khoảng 14 liên kết đổi trạng thái sau mỗi bước 20 giây.

Mỗi loại nút có vai trò riêng. Người dùng chỉ làm điểm đầu hoặc điểm cuối, không được chuyển tiếp gói tin. HAPS chỉ làm nút chuyển tiếp, không làm điểm đầu hay cuối. Các loại còn lại vừa chuyển tiếp được, vừa làm điểm đầu cuối được.

2.2. Bài toán định tuyến và thuật toán tham chiếu

Với đồ thị có hướng $G_t = (V, E_t, w_t)$ và một cặp nguồn s, đích d, ta cần tìm đường có tổng độ trễ nhỏ nhất trong tập $\mathscr{P}(s,d)$ các đường hợp lệ, tức là các đường không đi ra từ một nút không được chuyển tiếp, trừ khi đó chính là nguồn:

$$P^* = \operatorname{argmin}_{P \in \mathscr{P}(s,d)} \sum_{(u,v) \in P} w_t(u,v) \qquad (1)$$

Nhãn được tạo bằng thuật toán Dijkstra [12] cải biên: khi lấy một nút ra khỏi hàng đợi ưu tiên, nếu nút đó không được chuyển tiếp và không phải nguồn thì không mở rộng tiếp. Vì mọi trọng số dương, thuật toán cho lời giải tối ưu với độ phức tạp $O((|V| + |E|) \log |V|)$. Mô hình học của nhóm có nhiệm vụ bắt chước lời giải này, nên Dijkstra vừa là nguồn nhãn, vừa là mốc so sánh về chất lượng và thời gian.

2.3. Đưa về học có giám sát: chuỗi quyết định next-hop

Một đường $P = (v_0 = s, v_1, \ldots, v_K = d)$ được tách thành K quyết định. Ở bước k, đang ở nút $c = v_k$, mô hình phải chọn v_{k+1} trong tập ứng viên hợp lệ. Tập này được biểu diễn bằng mặt nạ M:

$$M_j = 1[W_t(c,j) > 0] \cdot 1[j = d \text{ hoặc } j \text{ chuyển tiếp được}] \cdot 1[j \text{ chưa đi qua}] \qquad (2)$$

Mặt nạ bảo đảm mọi quyết định đều hợp lệ về mặt vật lý: có liên kết, không đi vào người dùng không phải đích, không quay lại nút cũ. Khi suy luận, mô hình sinh cả đường bằng cách lặp quyết định từ nguồn, tối đa 12 bước. Nếu hết ứng viên hoặc hết số bước mà chưa tới đích thì tính là thất bại.

3. Dữ liệu

3.1. Sinh nhãn và cấu trúc tệp

Mỗi snapshot lấy 50 cặp nguồn – đích ngẫu nhiên trong các nút được làm điểm đầu cuối, tổng cộng 30.000 mẫu. Mỗi mẫu lưu chỉ số snapshot, đường Dijkstra dạng dãy chỉ

số nút, và dạng chữ để đọc. Ma trận trễ lưu một lần cho mỗi snapshot (`W_snapshots`), các mẫu tra cứu qua `snapshot_id`, nên tệp gọn: tập train 73 nút chỉ khoảng 0,8 MB.

3.2. Chia tập theo pha quỹ đạo

Nếu chia ngẫu nhiên theo mẫu, hai snapshot cách nhau 20 giây, gần như giống hệt nhau, có thể rơi vào hai tập khác nhau, khiến kết quả test bị thổi phồng. `split_dataset.py` chia theo pha: TV1 đo độ tương đồng Jaccard giữa tập cạnh của các snapshot và thấy cấu hình mạng lặp lại sau khoảng 141 snapshot (khoảng 47 phút), nên snapshot ở thời điểm t gần như trùng với snapshot ở t ± 47 phút. Vì vậy vị trí trong chu kỳ (pha) được chia thành 12 khối, các khối gán cho train, val và test, và bỏ 3 snapshot ở mỗi biên khối. Kết quả được liệt kê ở Bảng 1. Trong tập test, 722 đường đi qua vệ tinh và 1.978 đường chỉ dùng tầng không trung và mặt đất.

Bảng 1. Kích thước các tập dữ liệu (73 nút)

Tập	Snapshot	Số đường	Số quyết định next-hop
Train	315	15.750	47.856
Val	72	3.600	10.984
Test	54	2.700	8.155
Bỏ ở biên khối	159	–	–

3.3. Nhãn nhiều tuyến

Nhãn Dijkstra chỉ có một đường cho mỗi cặp, nên mô hình học được một phân phối gần như tập trung vào một điểm và không có cơ sở để sinh đường thay thế. Để đánh giá khả năng sinh nhiều tuyến, nhóm dùng thuật toán Yen [13] (có quy tắc chuyển tiếp) để tìm tối đa 4 đường đơn có trễ không quá 1,1 lần tối ưu. Trung bình mỗi cặp có 1,72 đường như vậy, và 38,9% số cặp có từ 2 đường trở lên. Mỗi epoch, mỗi cặp được gán ngẫu nhiên một trong các đường đó làm nhãn, nên mô hình học được một phân phối trên các đường gần tối ưu.

4. Cơ sở lý thuyết

4.1. Mạng nơ-ron đồ thị truyền tin

GNN học biểu diễn cho từng nút bằng cách lặp lại việc trao đổi thông tin với hàng xóm (message passing) [8, 9]. Đầu vào của nút i là vector 12 chiều: tầng (3 chiều one-hot), loại nút (5), cờ chuyển tiếp (1), bậc chuẩn hóa, trễ trung bình các cạnh kề, và cờ đánh dấu nút đích. Mỗi lớp của nhóm gom tin theo hai kênh: trung bình đều của hàng xóm, và trung bình ưu tiên cạnh trễ thấp với thang trễ τ học được:

$$\bar{A}_{ij} = A_{ij} / \deg(i), \quad B_{ij} = \exp(-w_{ij}/\tau) / \Sigma_k \exp(-w_{ik}/\tau) \tag{3}$$

$$a_i = \Sigma_j \bar{A}_{ij} W_m h_j^{(l)}, \quad b_i = \Sigma_j B_{ij} W_m h_j^{(l)}$$

$$h_i^{(l+1)} = \mathrm{LayerNorm}(h_i^{(l)} + \mathrm{MLP}([h_i^{(l)} \| a_i \| b_i])) \tag{4}$$

Kết nối tắt và LayerNorm [17] giữ cho việc huấn luyện ổn định. Phép gom bằng tổng có trọng số không phụ thuộc thứ tự đánh số nút, nên mô hình bất biến với hoán vị và

chạy được với số nút bất kỳ; bộ test đơn vị kiểm tra đúng tính chất này. Sau L lớp, biểu diễn của một nút chứa thông tin trong bán kính L bước nhảy.

Có một lý do lý thuyết để kỳ vọng GNN hợp với bài toán đường ngắn nhất. Thuật toán Bellman–Ford cập nhật khoảng cách tới đích theo d_i ← min_j (w_ij + d_j). Mỗi lần cập nhật chỉ dùng thông tin của hàng xóm, giống hệt cấu trúc một lớp truyền tin, chỉ khác phép min được thay bằng phép gom học được. Các nghiên cứu về căn chỉnh thuật toán cho thấy GNN học rất tốt những thuật toán có cấu trúc như vậy [15, 16]. Vì chỉ có cờ nút đích đi vào GNN, sau 4 lớp mỗi nút mang một biểu diễn kiểu "còn cách đích bao xa", đủ cho các đường trong dữ liệu (1–6 bước, trung bình 3 bước). GNN chỉ phụ thuộc vào snapshot và đích, nên chạy một lần cho cả đường, không phải chạy lại ở mỗi bước.

4.2. Mô hình khuếch tán khử nhiễu

Mô hình khuếch tán [1] định nghĩa một quá trình thuận làm nhiễu dần dữ liệu y_0 qua T bước, và học quá trình ngược để khử nhiễu. Nhờ tính chất của phân phối Gauss, có thể lấy mẫu trực tiếp ở bước t bất kỳ:

$$q(y_t \mid y_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\cdot y_0,\ (1-\bar{\alpha}_t)\,I), \quad \text{tức là}$$
$$y_t = \sqrt{\bar{\alpha}_t}\cdot y_0 + \sqrt{(1-\bar{\alpha}_t)}\cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I) \tag{5}$$

Nhóm dùng lịch cosine [3] với T = 100. Lịch này bảo đảm $\bar{\alpha}_T$ gần 0, tức là bước cuối gần như nhiễu thuần. Lịch tuyến tính thử lúc đầu không đạt điều này và làm mô hình học sai.

$$\bar{\alpha}_t = f(t)/f(0), \quad f(t) = \cos^2\!\left(\frac{t/T + s}{1+s}\cdot\frac{\pi}{2}\right), \quad s = 0{,}008 \tag{6}$$

4.3. Tham số hóa: dự đoán nhiễu hay dự đoán y0

Cách phổ biến trong sinh ảnh là cho mạng dự đoán nhiễu $\varepsilon_\theta(y_t, t, c)$ và tối thiểu sai số bình phương. Ở bài toán này, y_0 là vector one-hot trên N nút (1 tại next-hop đúng), và sai số chỉ tính trên các ứng viên hợp lệ:

$$L_\varepsilon = \mathbb{E}_{t,\varepsilon}\left[\ \| M \odot (\varepsilon - \varepsilon_\theta(y_t, t, c)) \|^2\ \right] / \|M\|_1 \tag{7}$$

Bản đầu tiên của nhóm ("DM cũ") dùng cách này. Tuy nhiên, mục tiêu thật của bài toán là chọn đúng một ứng viên, nên nhóm chuyển sang cho mạng xuất thẳng logit ℓ_i của y_0 và học bằng cross-entropy ("DM mới"), theo tinh thần của CDCD [5]:

$$\hat{y}_0 = \text{softmax_M}(\ell_\theta(y_t, t, c)), \qquad L_{CE} = -\log \hat{y}_0[j^*] \tag{8}$$

Ở đây softmax_M là softmax chỉ trên các ứng viên hợp lệ, và j* là next-hop đúng. Cross-entropy tối ưu thẳng xác suất chọn đúng. Ngược lại, sai số trên nhiễu là một mục tiêu gián tiếp, và phần lớn công sức học bị dồn vào các mức nhiễu thấp, nơi y_t đã gần như lộ đáp án. Thực nghiệm ở Mục 6 cho thấy cách mới tốt hơn rõ khi không có GNN (72,6% lên 81,5%). Nhóm cũng thử thêm self-conditioning [6] (đưa ước lượng y0 của bước trước vào làm đầu vào) nhưng không thấy cải thiện, nên không giữ trong mô hình cuối.

4.4. Lấy mẫu DDIM và tham số η

Khi suy luận, bắt đầu từ $y_T \sim \mathcal{N}(0, I)$ và đi ngược qua một dãy bước con $t > t'$ theo công thức DDIM [2]. Mỗi bước, mạng cho ước lượng $\hat{y}_0$, từ đó suy ra nhiễu tương ứng $\hat{\varepsilon}$ và bước tiếp theo:

$$\hat{\varepsilon} = (y_t - \sqrt{\bar{\alpha}_t} \cdot \hat{y}_0) / \sqrt{(1 - \bar{\alpha}_t)}$$

$$y_{t'} = \sqrt{\bar{\alpha}_{t'}} \cdot \hat{y}_0 + \sqrt{(1 - \bar{\alpha}_{t'} - \sigma_t^2)} \cdot \hat{\varepsilon} + \sigma_t \cdot z \quad (9)$$

$$\sigma_t = \eta \cdot \sqrt{((1 - \bar{\alpha}_{t'})/(1 - \bar{\alpha}_t))} \cdot \sqrt{(1 - \bar{\alpha}_t/\bar{\alpha}_{t'})}, \quad z \sim \mathcal{N}(0, I) \quad (10)$$

Với $\eta = 0$, quá trình là tất định: cùng nhiễu ban đầu luôn cho cùng kết quả, và có thể dùng rất ít bước. Với $\eta = 1$, mỗi bước thêm nhiễu mới, tương đương lấy mẫu DDPM gốc, nên kết quả đa dạng hơn. Next-hop là ứng viên có $\hat{y}_0$ lớn nhất ở bước cuối. Với bản DM mới, 1–2 bước DDIM là đủ (Mục 6.3).

4.5. Hướng dẫn không cần bộ phân loại (CFG)

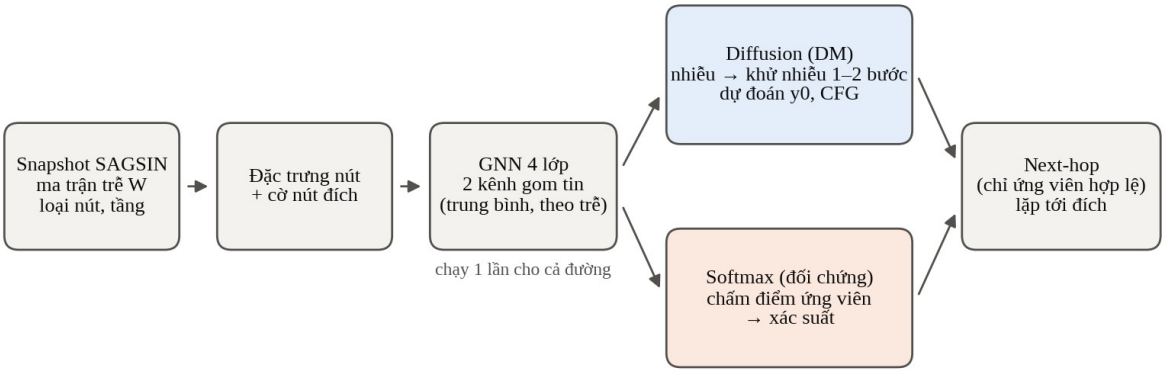
Khi huấn luyện, với xác suất 15%, cờ nút đích bị xóa khỏi cả đầu vào GNN lẫn bộ khử nhiễu, nên cùng một mạng học được cả mô hình có điều kiện ℓ_c lẫn mô hình không điều kiện $\ell_\varnothing$. Khi suy luận, hai đầu ra được trộn theo hệ số w [4]:

$$\ell_w = \ell_\varnothing + w \cdot (\ell_c - \ell_\varnothing) \quad (11)$$

$w = 1$ là mô hình có điều kiện thông thường, $w > 1$ khuếch đại ảnh hưởng của điều kiện, $w < 1$ làm mô hình bớt bám điều kiện và ra quyết định đa dạng hơn. w được chọn trên tập val, không nhìn tập test.

4.6. Áp dụng cho chọn next-hop

Hình 1 mô tả toàn bộ quy trình. Bộ khử nhiễu chấm điểm từng nút i. Nó ghép biểu diễn GNN của chính i, của nút hiện tại c và của đích d, cùng nhúng bước thời gian, với các đặc trưng cục bộ: giá trị nhiễu $y_{t,i}$, có cạnh $c \to i$ không, độ trễ cạnh đó, i có phải đích không, và trung bình, cực đại của y_t trên các ứng viên. Cách thiết kế này giống một bước Bellman–Ford: điểm của i dựa trên "i còn cách đích bao xa" (từ GNN) cộng với "đi từ c sang i tốn bao nhiêu" (trễ cạnh). Bản DM mới có 107.330 tham số, bản DM cũ 103.170.



Hình 1. Quy trình chọn next-hop. GNN chạy một lần cho cả đường; đầu ra là diffusion hoặc softmax (đối chứng); quyết định chỉ chọn trong ứng viên hợp lệ.

4.7. Mô hình đối chứng softmax

Để tách đóng góp của diffusion, mô hình đối chứng dùng cùng GNN và cùng đầu chấm điểm, nhưng bỏ các đầu vào y_t và t. Điểm $s_\theta(i)$ của các ứng viên được đưa qua softmax và học bằng cross-entropy, tức là một bộ phân loại thông thường (100.930 tham số):

$$p(j \mid c) = \exp(s_\theta(j)/T) \,/\, \Sigma_{k \in M} \exp(s_\theta(k)/T)$$

$$L = -\log p(j^* \mid c) \tag{12}$$

Khi suy luận, chọn ứng viên có điểm cao nhất, hoặc lấy mẫu với nhiệt độ T để sinh nhiều tuyến: T lớn thì phân phối phẳng hơn, kết quả đa dạng hơn. Như vậy, η và w đóng vai trò núm điều chỉnh độ đa dạng của DM, còn T là núm tương ứng của softmax.

4.8. Sinh nhiều tuyến và đường dự phòng

Với mỗi cặp, mô hình sinh K = 8 tuyến bằng 8 hạt nhiễu (DM) hoặc 8 lần lấy mẫu (softmax). Để đánh giá giá trị của các tuyến này, nhóm giả lập sự cố: lần lượt coi mỗi liên kết của tuyến chính là bị hỏng. Nếu đồ thị vẫn còn đường khác, ta kiểm tra trong 8 tuyến đã sinh có sẵn tuyến nào tránh được liên kết hỏng không, và so trễ của nó với Dijkstra chạy lại trên đồ thị đã bỏ liên kết đó. Chỉ số "có sẵn dự phòng" cho biết tỉ lệ tình huống mà bộ định tuyến có thể chuyển tuyến ngay, không cần tính lại.

5. Thiết kế thí nghiệm

Mọi mô hình dùng H = 64 chiều ẩn, AdamW [18] với tốc độ học $2 \cdot 10^{-3}$, suy giảm trọng số 10^{-4} và lịch OneCycle, mỗi lô 64 đường, 15 epoch, cắt chuẩn gradient ở 1,0. Checkpoint tốt nhất được chọn theo độ chính xác next-hop trên val. Với DM, số bước DDIM (1, 2, 5, 20) và hệ số w (1, 2, 4) cũng được chọn trên val. Hai mốc so sánh đơn giản là chọn ngẫu nhiên một ứng viên hợp lệ và chọn tham lam cạnh có trễ thấp nhất.

Các chỉ số trên tập test gồm:

- **chọn đúng next-hop**: tỉ lệ quyết định trùng Dijkstra khi đứng trên đường đúng;
- **tới đích**: tỉ lệ đường sinh ra tới được đích;
- **trễ / Dijkstra**: tổng trễ của đường sinh ra chia cho trễ tối ưu, chỉ tính trên các đường tới đích;
- **trong 10% tối ưu** và **trùng đường oracle**: độ gần với lời giải tối ưu;
- **thời gian**: mili giây cho mỗi đường, đo cả khi hỏi từng đường một và khi xử lý theo lô, trên CPU 2 luồng.

Lưu ý khi đọc bảng: các chỉ số trễ chỉ tính trên những đường tới được đích, nên một phương pháp ít tới đích có thể trông tốt giả tạo ở các cột này. Mỗi cấu hình chỉ chạy với một hạt giống ngẫu nhiên, nên chênh lệch dưới khoảng 0,5 điểm nên xem là ngang nhau.

6. Kết quả

6.1. Độ chính xác

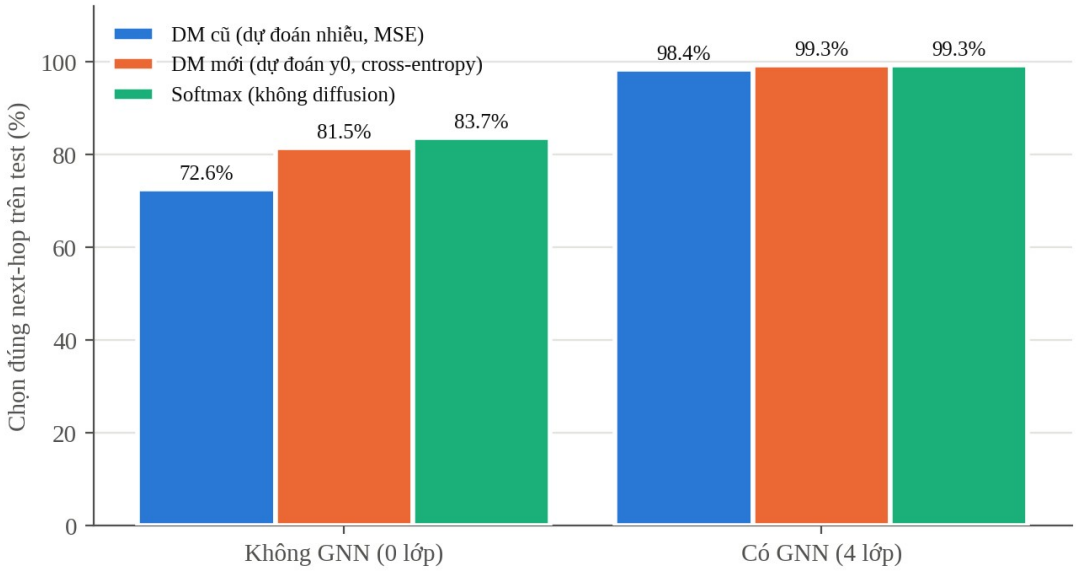
Bảng 2. Kết quả trên tập test (2.700 đường, 73 nút, snapshot chưa thấy khi huấn luyện)

Phương pháp	Next-hop	Tới đích	Trễ / Dijkstra	Trong 10%	Trùng oracle	Tham số
Ngẫu nhiên hợp lệ	41,1%	22,5%	1,663×	57,3%	12,0%	–
Tham lam trễ thấp	47,4%	25,6%	1,250×	69,0%	17,4%	–
DM cũ, không GNN	72,6%	81,0%	1,390×	60,3%	42,7%	19.970
DM mới, không GNN	81,5%	89,4%	1,235×	69,6%	57,4%	24.130
Softmax, không GNN	83,7%	91,2%	1,203×	74,3%	62,7%	17.730

Phương pháp	Next-hop	Tới đích	Trễ / Dijkstra	Trong 10%	Trùng oracle	Tham số
DM cũ, GNN 4 lớp	98,4%	100%	1,009×	97,9%	95,0%	103.170
DM mới, GNN 4 lớp	99,3%	100%	1,002×	99,3%	98,3%	107.330
Softmax, GNN 4 lớp	99,3%	100%	1,001×	99,5%	98,3%	100.930

Bản DM mới với GNN 4 lớp gần như trùng Dijkstra: 98,3% số đường trùng khớp hoàn toàn, và mọi đường đều tới đích. Kết quả đúng cho cả đường qua vệ tinh (722 đường, tới đích 100%, trễ 1,006×) lẫn đường chỉ dùng tầng không trung và mặt đất (1.978 đường, 100%, 1,000×). Mốc tham lam chỉ tới đích 25,6%. Điều này cho thấy nhìn cạnh kề là không đủ: phải có thông tin về cấu trúc toàn đường.

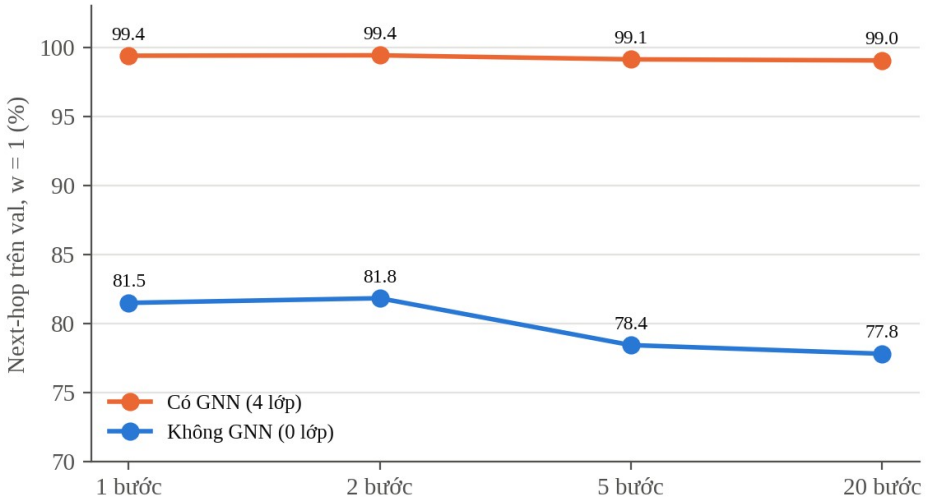
6.2. Đóng góp của GNN và của diffusion



Hình 2. Next-hop trên test theo lưới 2 × 2 (có/không GNN × loại đầu ra), kèm bản DM cũ.

Hình 2 tách hai yếu tố. Thêm GNN 4 lớp làm next-hop tăng 17,8 điểm với DM mới, 15,6 điểm với softmax và 25,8 điểm với DM cũ. Đổi softmax sang diffusion làm next-hop giảm 2,2 điểm khi không có GNN, và không đổi (0,0 điểm) khi có GNN. Nói cách khác, GNN là thành phần tạo ra độ chính xác, còn diffusion đạt ngang bộ phân loại khi được huấn luyện đúng cách. Cải tiến y0 + cross-entropy thu hẹp khoảng cách giữa diffusion và softmax khi không có GNN: từ 11,1 điểm (DM cũ so với softmax) xuống 2,2 điểm (DM mới so với softmax).

6.3. Số bước khử nhiễu và hệ số hướng dẫn



Hình 3. Next-hop trên val theo số bước DDIM của DM mới (w = 1).

Với DM mới, thêm bước khử nhiễu không giúp gì. Bản 4 lớp đạt 99,4% ở cả 1 và 2 bước, giảm nhẹ còn 99,0% ở 20 bước. Bản không GNN giảm rõ hơn, từ 81,8% ở 2 bước xuống 77,8% ở 20 bước. Nguyên nhân là mỗi quyết định chỉ là một lựa chọn rời rạc duy nhất: khi mạng đã cho ŷ_0 tốt ngay từ bước đầu, các bước sau chỉ mang theo nhiễu ban đầu và có thể làm lệch quyết định. CFG có ích với DM cũ (4 lớp: 97,2% ở w = 1 lên 98,4% ở w = 3; không GNN: 67,2% lên 71,9% ở w = 4). Với DM mới 4 lớp, w = 1 đã tốt nhất trên val (99,4%), còn w = 4 giảm còn 98,0%.

6.4. Thời gian

Bảng 3. Thời gian tìm một đường (ms, CPU 2 luồng, Python)

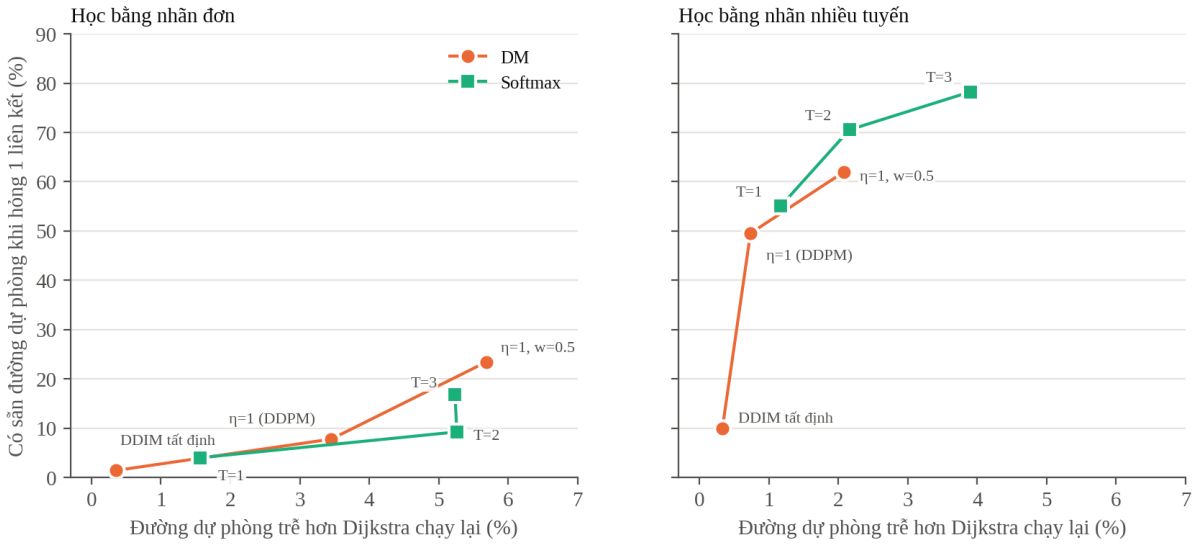
Phương pháp	Hỏi từng đường	Theo lô (ms/đường)
Dijkstra (Python, như oracle_labeler)	0,07	0,07
Tham lam trễ thấp	0,19	0,06
Softmax, GNN 4 lớp	2,82	0,53
DM mới 1 bước, GNN 4 lớp	3,94	0,52
DM mới 2 bước, GNN 4 lớp	5,22	0,61
DM 20 bước, GNN 4 lớp	21,96	1,74

Ở quy mô 73 nút, Dijkstra nhanh nhất: các đường chỉ dài trung bình 3 bước nên thuật toán dừng rất sớm. Mô hình học phải chạy mạng nơ-ron ở mỗi bước. DM 1–2 bước tốn ngang softmax; DM 20 bước chậm hơn khoảng 3–8 lần. Vì vậy báo cáo không khẳng định mô hình học nhanh hơn Dijkstra. Lợi thế có thể có của mô hình học trong SAGSIN nằm ở lượng thông tin cần biết (GNN k lớp chỉ cần thông tin trong k + 1 bước nhảy, trong khi Dijkstra cần bản đồ toàn mạng), nhưng điều này chưa được đo trong đồ án.

6.5. Sinh nhiều tuyến và đường dự phòng

Bảng 4. Sinh 8 tuyến mỗi cặp (600 cặp test, khoảng 980 tình huống hỏng một liên kết)

Mô hình	Cách lấy mẫu	Tuyến khác nhau / cặp	Có dự phòng	Trễ dự phòng / Dijkstra lại	ms cho 8 tuyến
DM, nhãn đơn	DDIM tất định	1,01	1,4%	1,004×	2,2
DM, nhãn đơn	η = 1, w = 0,5	1,28	23,5%	1,057×	24,7
Softmax, nhãn đơn	T = 3	1,15	16,8%	1,052×	1,5
DM, nhãn nhiều tuyến	DDIM tất định	1,09	10,0%	1,003×	2,1
DM, nhãn nhiều tuyến	η = 1	1,76	49,5%	1,007×	14,7
DM, nhãn nhiều tuyến	η = 1, w = 0,5	2,24	62,0%	1,021×	28,4
Softmax, nhãn nhiều tuyến	T = 1	1,94	55,1%	1,012×	1,5
Softmax, nhãn nhiều tuyến	T = 2	2,49	70,5%	1,022×	1,6
Softmax, nhãn nhiều tuyến	T = 3	2,88	78,3%	1,039×	1,6



Hình 4. Đánh đổi giữa tỉ lệ có sẵn đường dự phòng và chất lượng của nó. Mỗi điểm là một cách lấy mẫu.

Khi học bằng nhãn đơn, cả hai mô hình quá tự tin, 8 lần sinh gần như chỉ ra một tuyến, và tỉ lệ có sẵn dự phòng không quá 23,5%. Khi học bằng nhãn nhiều tuyến, độ chính xác gần như không đổi (DM: next-hop 97,7%, tới đích 100%, trễ 1,002×; softmax: 98,1%, 100%, 1,003×), nhưng mô hình sinh được nhiều tuyến hữu ích. DM với η = 1 có sẵn dự phòng cho 49,5% tình huống, và đường dự phòng chỉ trễ hơn Dijkstra chạy lại 0,7%. Tuy nhiên, ở cùng mức chất lượng dự phòng (khoảng 1,02×), softmax T = 2 phủ được 70,5% so với 62,0% của DM (η = 1, w = 0,5), và nhanh hơn khoảng 18 lần. Yếu tố quyết định khả năng sinh dự phòng là **cách xây dựng nhãn**, không phải loại mô hình sinh.

7. Thảo luận

7.1. Vai trò của diffusion trong mô hình

Từ các kết quả trên, vai trò của từng thành phần có thể phát biểu rõ ràng. GNN là bộ phận "hiểu mạng": nó học biểu diễn kiểu khoảng cách tới đích và quyết định gần như toàn bộ chất lượng định tuyến. Diffusion là **cơ chế sinh quyết định có điều kiện**. Nó đạt độ chính xác ngang bộ phân loại, đồng thời cung cấp hai núm điều khiển lúc suy luận mà không cần huấn luyện lại: hệ số w của CFG (mức bám thông tin đích) và η (độ ngẫu nhiên), cho phép sinh nhiều tuyến ứng viên. Cái giá phải trả là thời gian suy luận cao hơn khi dùng nhiều bước.

Nhóm không khẳng định diffusion làm tăng độ chính xác. Lý do lý thuyết là mỗi quyết định next-hop chỉ là một biến rời rạc với trung bình khoảng 5 ứng viên hợp lệ (tối đa 27). Với một biến như vậy, softmax với cross-entropy đã biểu diễn chính xác mọi phân phối và tối ưu trực tiếp mục tiêu; diffusion không có thêm khả năng biểu diễn nào. Thế mạnh của diffusion là mô hình hóa đồng thời nhiều biến phụ thuộc nhau, điều không xảy ra khi bài toán đã được tách thành từng bước.

7.2. So sánh với DIFUSCO

Bảng 5. So sánh với DIFUSCO (Sun & Yang, NeurIPS 2023)

Khía cạnh	DIFUSCO [7]	Đồ án này
Bài toán	TSP, MIS — NP-khó, đồ thị tĩnh	Đường trễ nhỏ nhất — giải chính xác được bằng Dijkstra, đồ thị động 3 tầng
Điều kiện	Không có (một đồ thị cho một lời giải)	Có: nút đích (CFG bỏ điều kiện đích)
Diffusion sinh gì	Cả heatmap trên mọi cạnh cùng lúc	Một lựa chọn next-hop mỗi bước
Bộ mã hóa	GNN bất đẳng hướng 12 lớp, rộng 256	GNN 4 lớp, rộng 64, 2 kênh gom tin
Giải mã	Greedy hoặc lấy mẫu + 2-opt, MCTS	Argmax trong mặt nạ hợp lệ, lặp tới đích
Kết quả chính	Greedy: gap 0,10% (TSP-50), 0,24% (TSP-100)	Trễ 1,002× Dijkstra, next-hop 99,3%

Hai bài đặt diffusion vào hai vai trò khác nhau. DIFUSCO cho diffusion sinh toàn bộ lời giải cùng lúc trên một không gian lớn theo cấp số mũ, và mô hình hóa sự phụ thuộc giữa hàng nghìn cạnh chính là thế mạnh của nó. Trên TSP-500 và TSP-1000, khi kết hợp tìm kiếm MCTS, DIFUSCO giảm khoảng cách tới tối ưu từ 1,76% xuống 0,46% và từ 2,46% xuống 1,17% so với phương pháp tốt nhất trước đó. Đồ án này tách bài toán thành chuỗi lựa chọn đơn, nên diffusion chỉ phải giải một bài toán mà softmax đã giải trọn vẹn. Bộ mã hóa của DIFUSCO kế thừa GNN bất đẳng hướng của Joshi và cộng sự [10]; DIFUSCO cũng không thí nghiệm bỏ GNN vì xem bộ mã hóa đồ thị là bắt buộc; đồ án này đo được đóng góp của nó nhờ mạng đủ nhỏ. Hai kết quả vì vậy không mâu thuẫn nhau.

7.3. Hạn chế

- Mạng mô phỏng 73 nút với mô hình trễ truyền theo khoảng cách; chưa có hàng đợi, tắc nghẽn hay suy hao kênh.
- Nhãn là Dijkstra, nên mô hình chỉ bắt chước, không vượt được lời giải tối ưu.
- Mỗi cấu hình chạy một hạt giống; chưa có khoảng tin cậy.
- Thời gian đo bằng Python trên CPU; chưa có cài đặt tối ưu trên GPU hay thiết bị bay.
- Chưa đo lượng thông tin phải trao đổi giữa các nút, là chỗ mô hình học có thể có lợi thế thật so với Dijkstra.

8. Kết luận và hướng phát triển

Đồ án đã phục dựng thành công một mô hình khuếch tán kết hợp GNN cho định tuyến trong SAGSIN. Mô hình đạt độ chính xác gần như tuyệt đối trên snapshot chưa thấy, và được đánh giá bằng thí nghiệm đối chứng trung thực. Kết luận chính: GNN quyết định chất lượng định tuyến; diffusion đạt ngang bộ phân loại khi được tham số hóa đúng (dự đoán y0, cross-entropy, 1–2 bước) và đóng vai trò bộ sinh quyết định có điều khiển; khả năng sinh đường dự phòng phụ thuộc chủ yếu vào việc huấn luyện bằng nhãn nhiều tuyến.

Hướng phát triển gồm: cho diffusion sinh cả tuyến cùng lúc trên heatmap cạnh, là vai trò mà bộ phân loại từng bước không thay thế được; đặt bài toán ở dạng Dijkstra không giải trực tiếp (nhiều tiêu chí, nhiều luồng chia sẻ băng thông); đo lượng thông tin cần trao đổi khi chạy phân tán; và thử nghiệm hỏng tầng để kiểm tra khả năng tổng quát hóa. Phụ lục B tóm tắt một thử nghiệm đầu theo hướng thứ hai.

Tài liệu tham khảo

[1] J. Ho, A. Jain, P. Abbeel, "Denoising Diffusion Probabilistic Models," NeurIPS, 2020.

[2] J. Song, C. Meng, S. Ermon, "Denoising Diffusion Implicit Models," ICLR, 2021.

[3] A. Nichol, P. Dhariwal, "Improved Denoising Diffusion Probabilistic Models," ICML, 2021.

[4] J. Ho, T. Salimans, "Classifier-Free Diffusion Guidance," arXiv:2207.12598, 2022.

[5] S. Dieleman và cộng sự, "Continuous diffusion for categorical data," arXiv:2211.15089, 2022.

[6] T. Chen, R. Zhang, G. Hinton, "Analog Bits: Generating Discrete Data using Diffusion Models with Self-Conditioning," ICLR, 2023.

[7] Z. Sun, Y. Yang, "DIFUSCO: Graph-based Diffusion Solvers for Combinatorial Optimization," NeurIPS, 2023. arXiv:2302.08224.

[8] J. Gilmer và cộng sự, "Neural Message Passing for Quantum Chemistry," ICML, 2017.

[9] T. N. Kipf, M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," ICLR, 2017.

[10] C. K. Joshi, T. Laurent, X. Bresson, "An Efficient Graph Convolutional Network Technique for the Travelling Salesman Problem," arXiv:1906.01227, 2019.

[11] J. Liu, Y. Shi, Z. M. Fadlullah, N. Kato, "Space-Air-Ground Integrated Network: A Survey," IEEE Communications Surveys & Tutorials, 20(4), 2018.

[12] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, 1, 1959.

[13] J. Y. Yen, "Finding the K Shortest Loopless Paths in a Network," Management Science, 17(11), 1971.

[14] A. Jüttner, B. Szviatovszki, I. Mécs, Z. Rajkó, "Lagrange relaxation based method for the QoS routing problem," IEEE INFOCOM, 2001.

[15] K. Xu và cộng sự, "What Can Neural Networks Reason About?," ICLR, 2020.

[16] P. Veličković và cộng sự, "Neural Execution of Graph Algorithms," ICLR, 2020.

[17] J. L. Ba, J. R. Kiros, G. E. Hinton, "Layer Normalization," arXiv:1607.06450, 2016.

[18] I. Loshchilov, F. Hutter, "Decoupled Weight Decay Regularization," ICLR, 2019.

Phụ lục A. Mã nguồn và cách chạy

Mã nguồn nằm trong thư mục `src/ai` và `test` của repo. Các tệp chính:

- `data_v2.py`: đọc dữ liệu 3 tầng, tạo đặc trưng nút, mặt nạ ứng viên, lô dữ liệu theo đường;
- `gnn_diffusion.py`: bộ mã hóa GNN, DM cũ (dự đoán nhiễu);
- `gnn_diffusion_v2.py`: DM mới (dự đoán y0, cross-entropy, lấy mẫu với η);
- `gnn_classifier.py`: mô hình đối chứng softmax;
- `train_gnn.py`: huấn luyện và đánh giá mọi biến thể qua tham số dòng lệnh;
- `nhieu_tuyen.py`: thuật toán Yen, sinh K tuyến, đánh giá đường dự phòng;
- `test/test_gnn.py` (18 test), `test/so_sanh_dm_softmax.py`, `test/so_sanh_gnn.py`, `test/do_thoi_gian.py`.

```
python test/test_gnn.py                          # 18 test, khoảng 15 giây
python test/so_sanh_dm_softmax.py                # huấn luyện nếu thiếu, in 3
bảng so sánh
python test/so_sanh_gnn.py                       # lưới có/không GNN
python test/do_thoi_gian.py                      # đo thời gian
python src/ai/ve_hinh_bao_cao.py                 # vẽ lại các hình của báo cáo

# huấn luyện riêng
python src/ai/train_gnn.py --head diffusion2 --param x0ce --layers 4 --ddim 2
--tag dmx0_4lop
python src/ai/train_gnn.py --head classifier --layers 4 --tag clsB_73
python src/ai/train_gnn.py --head diffusion2 --param x0ce --layers 4 --ddim 2
--nhieu-tuyen 0.1 --tag dm_nt
```

Phụ lục B. Thử nghiệm mở rộng: định tuyến có ràng buộc độ tin cậy

Để thử đặt diffusion vào bài toán mà Dijkstra không giải trực tiếp, nhóm xây bài toán "trễ thấp nhất với độ tin cậy của đường ≥ p". Độ tin cậy của liên kết là $p_{ij} = 1 - \exp(-L_{ij}/T0)$, với L_{ij} là thời gian liên kết còn tồn tại (đọc từ chuỗi snapshot) và T0 =

120 giây. Độ tin cậy của đường là tích các p_ij. Đây là bài toán đường ngắn nhất có ràng buộc tài nguyên, NP-khó. Nhãn tối ưu được tính bằng label-setting hai tiêu chí và đã kiểm tra khớp với vét cạn. Trên nền 73 nút gần như không có đánh đổi giữa trễ và độ tin cậy (chỉ khoảng 1,5% số cặp có từ 2 đường Pareto), nên thử nghiệm dùng nền 173 nút (150 vệ tinh).

Một mô hình diffusion sinh cả tuyến trên heatmap cạnh, có điều kiện theo (nguồn, đích, p), phục vụ được mọi mức p và thật sự điều chỉnh tuyến theo p. Khi tắt kiểm tra khả thi ở khâu giải mã, độ tin cậy trung bình của tuyến tăng từ 0,60 lên 0,97 khi p tăng từ 0,5 lên 0,95. Tuy vậy, trên nhóm truy vấn mà Dijkstra vi phạm ràng buộc, diffusion chọn đúng đường tối ưu 77,0% so với 78,3% của bản không diffusion; khi lấy 8 mẫu là 91,7% so với 92,5%. Heuristic LARAC [14] đạt 90,6% với luôn đúng ràng buộc và chỉ mất khoảng 1 ms. Kết quả này nhất quán với phần chính: chất lượng đến từ GNN, cách đặt điều kiện và cách giải mã, không đến từ quá trình khuếch tán.